



Programa educativo
INGENIERÍA EN SISTEMAS
COMPUTACIONALES

Grupo
8 ° B

Nombre de la materia
ARQUITECTURA DE SERVICIOS

Nombre del alumno
SANTOS GÓMEZ FERNANDO
ARCINIEGA ROBLES FRANCISCO DANIEL
RAMIREZ MENDOZA ANGEL RAMON

Nombre del Profesor
ROBERTO SUAREZ ZINZUN

Fecha
31/05/2026

POLÍTICA DE SEGURIDAD PARA LA API REST DEL SISTEMA DE GESTIÓN DE EVENTOS Y ACTIVIDADES

Objetivo

El presente documento tiene como objetivo definir la política de seguridad de la API REST del sistema de gestión de eventos y actividades para alumnos. Esta política establece los roles de usuario, los permisos de acceso a la base de datos y a los endpoints de la API, así como las reglas generales de autenticación, autorización y limitación de solicitudes.

Tecnologías utilizadas

- Python.
- FastAPI.
- MongoDB.
- JSON Web Token (JWT) para autenticación.
- Passlib con bcrypt para cifrado de contraseñas.
- SlowAPI para la configuración de rate limit.

Roles del sistema

En el sistema se definieron los siguientes roles de usuario:

Administrador

Usuario con privilegios de administración del sistema. Puede gestionar alumnos, carreras, grupos e inscripciones administrativas, además de consultar y mantener información general del sistema.

Organizador

Usuario encargado de gestionar eventos y actividades. Tiene permisos para crear, actualizar y cancelar eventos, así como crear, actualizar y eliminar actividades.

Alumno

Usuario final del sistema. Puede autenticarse en la plataforma, inscribirse a eventos y cancelar sus inscripciones.

Política de seguridad a nivel de base de datos

La base de datos deberá implementar privilegios diferenciados de acuerdo con el rol del usuario.

Permisos del Administrador

El administrador podrá realizar operaciones de consulta, inserción, actualización y eliminación sobre las colecciones principales del sistema, como alumnos, carreras, grupos, eventos, actividades e inscripciones.

Permisos del Organizador

El organizador podrá consultar información necesaria para su operación y gestionar principalmente eventos y actividades. También podrá consultar información relacionada con el sistema para fines operativos.

Permisos del Alumno

El alumno tendrá permisos restringidos. Su acceso se limita a la consulta de información general disponible y a la gestión de sus inscripciones, de acuerdo con las reglas definidas en la API.

Política de seguridad de la API REST

La autorización de operaciones en la API REST se basa en roles. Cada endpoint protegido valida el token del usuario autenticado y su rol antes de ejecutar la operación.

Recurso Alumnos

- **POST /alumnos:** permitido solo para Administrador.
- **POST /alumnos/login:** permitido para autenticación de usuarios.
- **GET /alumnos:** permitido solo para Administrador.
- **GET /alumnos/{idAlumno}:** permitido solo para Administrador.
- **PATCH /alumnos/{idAlumno}/activar:** permitido solo para Administrador.
- **PATCH /alumnos/{idAlumno}/desactivar:** permitido solo para Administrador.
- **DELETE /alumnos/{idAlumno}:** permitido solo para Administrador.

Recurso Carreras

- **POST /carreras:** permitido solo para Administrador.
- **GET /carreras:** permitido solo para Administrador.
- **GET /carreras/{idCarrera}:** permitido solo para Administrador.
- **PUT /carreras/{idCarrera}:** permitido solo para Administrador.
- **PATCH /carreras/{idCarrera}/activar:** permitido solo para Administrador.
- **PATCH /carreras/{idCarrera}/desactivar:** permitido solo para Administrador.
- **DELETE /carreras/{idCarrera}:** permitido solo para Administrador.

Recurso Grupos

- **POST /grupos:** permitido solo para Administrador.
- **GET /grupos:** permitido solo para Administrador.
- **GET /grupos/{idGrupo}:** permitido solo para Administrador.
- **PUT /grupos/{idGrupo}:** permitido solo para Administrador.
- **PATCH /grupos/{idGrupo}/activar:** permitido solo para Administrador.
- **PATCH /grupos/{idGrupo}/desactivar:** permitido solo para Administrador.
- **DELETE /grupos/{idGrupo}:** permitido solo para Administrador.

Recurso Eventos

- **POST /eventos/:** permitido para Organizador.
- **GET /eventos/:** consulta general disponible.
- **GET /eventos/{idEvento}:** consulta disponible.
- **PUT /eventos/{idEvento}:** permitido para Organizador.
- **PATCH /eventos/{idEvento}/cancelar:** permitido para Organizador.

Recurso Actividades

- **POST /actividades:** permitido para Organizador.
- **GET /actividades:** consulta disponible.
- **GET /actividades/{idActividad}:** consulta disponible.
- **PUT /actividades/{idActividad}:** permitido para Organizador.
- **DELETE /actividades/{idActividad}:** permitido para Organizador.

Recurso Inscripciones

- **POST /inscripciones/:** permitido para Alumno.
- **GET /inscripciones/:** permitido solo para Administrador en la implementación actual.
- **GET /inscripciones/{idInscripcion}:** permitido solo para Administrador en la implementación actual.
- **DELETE /inscripciones/{idInscripcion}:** permitido para Alumno.

Reglas de autenticación

La autenticación de usuarios se implementa mediante JSON Web Token (JWT).

Después de validar credenciales, el sistema genera un token de acceso que contiene información del usuario autenticado, incluyendo su rol. Dicho token debe enviarse en las rutas protegidas para acceder a los recursos autorizados.

Las contraseñas no se almacenan en texto plano. Se protegen mediante hash usando bcrypt a través de la librería Passlib.

Reglas de autorización

La autorización se implementa mediante validación de roles dentro de la API REST.

Cada operación protegida revisa el rol del usuario autenticado y permite o rechaza el acceso según los permisos definidos para el endpoint correspondiente.

En caso de no enviar token válido, la API responde con error de autenticación. En caso de enviar un token válido pero con rol insuficiente, la API responde con error de autorización.

Política de rate limit

Con el objetivo de evitar abuso del servicio, intentos masivos de autenticación y sobrecarga de la API, se implementó una política de limitación de solicitudes por dirección IP utilizando SlowAPI.

Los límites configurados en la implementación actual son los siguientes:

- Login: 5 solicitudes por minuto.
- Operaciones de escritura (POST, PUT, PATCH, DELETE): 10 solicitudes por minuto.
- Operaciones de consulta (GET) en algunos recursos administrativos: 20 solicitudes por minuto.
- Operaciones de consulta (GET) en recursos como eventos y actividades: 30 solicitudes por minuto.

Cuando un cliente excede el número de solicitudes permitido, la API responde con el código HTTP 429, indicando exceso de peticiones.

Reglas generales de seguridad

- Toda operación protegida debe validar autenticación y rol antes de ejecutarse.
- Las contraseñas deben almacenarse con hash seguro y nunca en texto plano.
- El acceso a recursos administrativos debe limitarse al rol Administrador.
- El acceso a la gestión de eventos y actividades debe limitarse al rol Organizador.
- El acceso a la creación y cancelación de inscripciones debe limitarse al rol Alumno.
- La API debe aplicar rate limit para prevenir abuso y uso excesivo del servicio.

Resultado esperado

Con esta política de seguridad, el sistema establece control de acceso por roles, protección de credenciales, autenticación mediante tokens, autorización por endpoint y limitación de solicitudes, fortaleciendo así la seguridad general de la API REST del proyecto.

Script para roles de MongoDB

```
use gestion_eventos_db;
```

```
db.createRole({
  role: "administrador_eventos",
  privileges: [
    {
      resource: { db: "gestion_eventos_db", collection: "" },
      actions: ["find", "insert", "update", "remove"]
    }
  ],
  roles: []
});
```

```
db.createRole({
  role: "organizador_eventos",
  privileges: [
    {
      resource: { db: "gestion_eventos_db", collection: "eventos" },
      actions: ["find", "insert", "update"]
    },
    {
      resource: { db: "gestion_eventos_db", collection: "actividades" },
      actions: ["find", "insert", "update", "remove"]
    }
  ],
  roles: []
});
```

```
db.createRole({
  role: "alumno_eventos",
  privileges: [
```

```
{
  resource: { db: "gestion_eventos_db", collection: "inscripciones" },
  actions: ["find", "insert", "remove"]
}
],
roles: []
});
```

Script para crear usuarios de prueba

```
use gestion_eventos_db;
```

```
db.alumnos.insertMany([
  {
    nombre: "Admin Usuario",
    email: "admin@itesz.edu.mx",
    password: "$2b$12$LQv3c1yqBWVHxkd0LHakCOYz6TtxMQJqhN8/LewY5GyYIr9oIWe7e",
    numeroControl: "ADMIN001",
    rol: "admin",
    tipo: "administrador",
    semestre: 0,
    estatus: true,
    fechaCreacion: new Date()
  },
  {
    nombre: "Organizador Usuario",
    email: "organizador@itesz.edu.mx",
    password: "$2b$12$LQv3c1yqBWVHxkd0LHakCOYz6TtxMQJqhN8/LewY5GyYIr9oIWe7e",
    numeroControl: "ORG001",
    rol: "organizador",
    tipo: "organizador",
    semestre: 0,
    estatus: true,
    fechaCreacion: new Date()
  }
]);
```

```
},  
{  
  nombre: "Alumno Prueba",  
  email: "alumno@itesz.edu.mx",  
  password: "$2b$12$LQv3c1yqBWVHxkd0LHakCOYz6TtxMQJqhN8/LewY56yYIr9oIWe7e",  
  numeroControl: "20240001",  
  rol: "alumno",  
  tipo: "regular",  
  semestre: 8,  
  estatus: true,  
  fechaCreacion: new Date()  
}  
]);  
  
print("Usuarios creados. Password para todos: password123");
```

Instrucciones de cómo usar los scripts

Crear usuarios de prueba

En la terminal:

```
mongosh < crear_usuarios_prueba.js
```

Usuarios disponibles

- Admin: admin@itesz.edu.mx / password123
- Organizador: organizador@itesz.edu.mx / password123
- Alumno: alumno@itesz.edu.mx / password123